

Self-Study Learning Outline: Machine Learning & AI

Modeled on the UC Berkeley Professional Certificate in ML/AI (24 modules, ~6 months)

Companion text: Practical Statistics for Data Scientists, 2nd ed. (Bruce, Bruce & Gedeck) — abbreviated PSDS

How to use this outline

Each module below is designed for ~1 week of focused study (8-10 hours). For each module you get:

- **Concepts** — what to learn and in what order
- **Reading** — chapters from PSDS (and slots for your other books)
- **Practice** — hands-on tasks to cement the concept
- **Self-check** — what you should be able to do before moving on

Don't skip the "Practice" sections. The Berkeley program's value is in its assignments, not its lectures, and that's what you need to replicate. Push every notebook to GitHub from Module 1 onward — by Week 24 you'll have a portfolio without having to assemble one.

Phase 0: Prerequisites (Weeks 0-2)

Before Module 1. Skip what you already know cold.

Math refresher (target: comfortable, not fluent)

- Linear algebra: vectors, matrices, dot products, eigenvalues/eigenvectors. Use 3Blue1Brown's *Essence of Linear Algebra* (YouTube, ~3 hours).
- Calculus: partial derivatives, the chain rule, gradients. 3Blue1Brown's *Essence of Calculus*.
- Probability: random variables, distributions, conditional probability, Bayes' rule.

Python toolchain

- Install Anaconda, set up Jupyter, create a GitHub account, learn the basic `git` workflow (`add`, `commit`, `push`, `pull`).
- Get fluent with: `numpy`, `pandas`, `matplotlib`. Aim for the level where you don't need to look up basic indexing or groupby.

Self-check: You can load a CSV with pandas, compute summary stats, plot a histogram, and push the notebook to a public GitHub repo — all without searching.

PART 1 — Foundations of ML/AI (Weeks 1-5)

Module 1: Introduction to Machine Learning

- **Concepts:** Supervised vs. unsupervised vs. reinforcement learning. Regression vs. classification. The data science lifecycle (CRISP-DM or similar). Bias-variance tradeoff at an intuitive level.
- **Reading:** PSDS — skim the Preface and Ch. 1 intro to get the book's vocabulary.
- **Practice:** Pick one Kaggle "starter" dataset (Titanic or Ames Housing). Don't model yet — just write a one-page document describing what problem you'd solve, what variables you have, and what success would look like.
- **Self-check:** You can explain to a non-technical friend the difference between prediction and inference.

Module 2: Fundamentals of Statistics and Distribution Functions

- **Concepts:** Mean, median, variance, standard deviation. Normal, binomial, Poisson distributions. Central Limit Theorem. Sampling and standard error. Confidence intervals.
- **Reading:** PSDS Ch. 1 (Exploratory Data Analysis) and PSDS Ch. 2 (Data and Sampling Distributions) — these are foundational; read carefully.
- **Practice:** In a Jupyter notebook, generate samples from each distribution, plot them, and demonstrate the CLT empirically (sample means of any distribution → normal). Compute a 95% confidence interval by hand and verify with `scipy.stats`.
- **Self-check:** You can articulate why a confidence interval is *not* "a 95% probability the true mean is in this range."

Module 3: Introduction to Data Analytics

- **Concepts:** Hypothesis testing, p-values, t-tests, chi-square tests. Type I/II errors. A/B testing logic.
- **Reading:** PSDS Ch. 3 (Statistical Experiments and Significance Testing) — every section.
- **Practice:** Run a two-sample t-test on a real dataset (e.g., compare male vs. female fare prices on Titanic). Explain the result in plain English.

- **Self-check:** You can describe what a p-value actually means and why " $p < 0.05$ " is a convention, not a law.

Module 4: Fundamentals of Data Analytics

- **Concepts:** EDA workflows, data cleaning, handling missing values, outlier detection, data visualization for insight (not decoration).
- **Reading:** Re-read PSDS Ch. 1 with practice in mind. Pay special attention to robust estimates (median, MAD, trimmed mean) and the distinction between exploratory and confirmatory plotting.
- **Practice:** Take a messy real dataset (try the NYC Taxi data, or any Kaggle dataset with data quality issues). Produce a clean version + a written EDA report with at least 8 charts that each answer a specific question.
- **Self-check:** Your charts have titles, axis labels, and a sentence underneath explaining what to take away.

Module 5: Practical Application I — *First Mini-Project*

- **Goal:** Apply Modules 1-4 end-to-end with no new theory.
 - **Deliverable:** A GitHub repo with a clean README containing: problem statement, dataset description, EDA findings, statistical tests run, and conclusions. No modeling yet — this proves you can do the analyst half of data science.
-

PART 2 — ML/AI Techniques (Weeks 6-12)

Module 6: Clustering and Principal Component Analysis

- **Concepts:** k-means, hierarchical clustering, evaluating clusters (silhouette score). PCA as dimensionality reduction; eigenvectors as principal components.
- **Reading:** PSDS Ch. 7 (**Unsupervised Learning**) — full chapter.
- **Practice:** Apply k-means to a customer segmentation dataset. Use PCA to reduce a high-dimensional dataset to 2D and visualize the clusters. Use the elbow method to choose k .
- **Self-check:** You can explain why PCA components are linear combinations of original features, and why scaling matters before running it.

Module 7: Linear and Multiple Regressions

- **Concepts:** Simple and multiple linear regression. Coefficients, R^2 , residual analysis.

Assumptions (linearity, homoscedasticity, normal residuals, independence) and how to check them.

- **Reading:** PSDS Ch. 4 (**Regression and Prediction**) — sections on simple/multiple regression and prediction vs. explanation. *Read this twice.*
- **Practice:** Build a regression model on the Boston Housing or Ames Housing dataset using both `statsmodels` (for inference) and `scikit-learn` (for prediction). Inspect residual plots.
- **Self-check:** Given a regression coefficient, you can explain what it means in plain English including units.

Module 8: Feature Engineering and Overfitting

- **Concepts:** One-hot encoding, scaling, polynomial features, interaction terms. Train/validation/test splits. Cross-validation. Overfitting vs. underfitting (visualize the curves).
- **Reading:** PSDS Ch. 4 — the sections on factor variables, interpreting coefficients, and partial residual plots.
- **Practice:** Take your Module 7 model. Engineer at least 3 new features. Run 5-fold cross-validation and plot training vs. validation error as model complexity grows.
- **Self-check:** You can sketch the classic overfitting curve from memory and explain where the "best" model lives on it.

Module 9: Model Selection and Regularization

- **Concepts:** Ridge (L2), Lasso (L1), Elastic Net. Why regularization works. Hyperparameter tuning with grid/random search. Bias-variance decomposition formally.
- **Reading:** PSDS Ch. 4 — regularization section.
- **Practice:** Run Ridge and Lasso on a high-dimensional dataset. Plot the coefficient paths as the regularization strength increases. Identify which features Lasso zeroes out.
- **Self-check:** You can explain why Lasso produces sparse solutions and Ridge doesn't.

Module 10: Time Series Analysis and Forecasting

- **Concepts:** Trend, seasonality, stationarity. Autocorrelation, ACF/PACF plots. Moving averages, exponential smoothing, ARIMA at an applied level.
- **Reading:** PSDS does not cover this — **slot for companion book**. Hyndman & Athanasopoulos's *Forecasting: Principles and Practice* (free online at otexts.com/fpp3) is the standard.

- **Practice:** Forecast monthly airline passengers (classic dataset) or any Yahoo Finance time series. Compare a naive baseline, exponential smoothing, and ARIMA.
- **Self-check:** You always check for stationarity before fitting an ARIMA, and you know what differencing does.

Module 11: Practical Application II — *Second Mini-Project*

- **Goal:** Build a regression problem end-to-end with regularization and proper validation.
- **Deliverable:** GitHub repo with a Kaggle-style writeup. Include feature engineering decisions, model comparison table, and a section on what you would do differently with more time.

Module 12: Classification and k-Nearest Neighbors

- **Concepts:** Classification metrics — accuracy, precision, recall, F1, ROC-AUC. Confusion matrix. KNN algorithm and the curse of dimensionality.
 - **Reading:** PSDS Ch. 5 (Classification) intro and KNN sections, plus PSDS Ch. 6 for KNN as a "statistical ML" method.
 - **Practice:** Build a KNN classifier on a binary classification problem. Compute and interpret the full confusion matrix. Plot the ROC curve and explain what each point on it represents.
 - **Self-check:** You can recommend the right metric (precision vs. recall vs. F1 vs. AUC) for a given business context — e.g., spam filter vs. cancer screen.
-

PART 3 — Advanced ML/AI (Weeks 13–22)

Module 13: Logistic Regression

- **Concepts:** Log-odds, the logit link, maximum likelihood estimation. Interpreting coefficients as odds ratios. Class imbalance and how to handle it.
- **Reading:** PSDS Ch. 5 — full chapter, especially logistic regression and evaluating classification models.
- **Practice:** Predict customer churn with logistic regression. Convert coefficients to odds ratios and explain what each predictor does.
- **Self-check:** You can explain why logistic regression is called "regression" even though it's a classifier.

Module 14: Decision Trees

- **Concepts:** How a tree splits (Gini impurity, entropy/information gain). Tree depth and overfitting. Pruning. Why trees handle non-linearity natively.
- **Reading:** PSDS Ch. 6 — tree models section.
- **Practice:** Train a decision tree, visualize it (graphviz or `sklearn.tree.plot_tree`), and walk through 3 example predictions by hand following the tree.
- **Self-check:** You can explain why a single decision tree is high-variance and what that implies for ensemble methods.

Module 15: Gradient Descent and Optimization

- **Concepts:** Cost functions, gradients, learning rate. Batch, stochastic, and mini-batch gradient descent. Why optimization is the engine inside almost every ML model.
- **Reading:** PSDS doesn't cover this in depth — **slot for companion book**. Géron's *Hands-On ML* Ch. 4 is excellent here (framework-agnostic — uses sklearn).
- **Practice:** Implement gradient descent from scratch in numpy to fit a linear regression. Plot the loss curve. Then break it deliberately by setting the learning rate too high and observe the divergence.
- **Self-check:** You can sketch a 2D loss surface and the path gradient descent takes across it.

Module 16: Classifying Nonlinear Features

- **Concepts:** Kernel methods, Support Vector Machines (linear and RBF kernel), the kernel trick at an intuitive level.
- **Reading:** PSDS doesn't cover SVMs — **companion slot**. Géron Ch. 5 or ISLR Ch. 9.
- **Practice:** Train an SVM on a non-linearly-separable 2D dataset (sklearn's `make_moons`). Visualize the decision boundary with linear vs. RBF kernels.
- **Self-check:** You can describe what the kernel trick is doing without writing math.

Module 17: Practical Application III — *Third Mini-Project*

- **Goal:** Classification problem, end-to-end, with model comparison.
- **Deliverable:** Pick a problem (fraud detection, customer churn, medical diagnosis dataset). Compare logistic regression, decision tree, KNN, and SVM. Justify your final choice on metric grounds, not accuracy alone.

Module 18: Natural Language Processing

- **Concepts:** Tokenization, stopwords, stemming/lemmatization. Bag-of-words, TF-IDF. Word embeddings (Word2Vec, GloVe) at a conceptual level. Sentiment analysis as a canonical task.
- **Reading:** PSDS doesn't cover NLP — **companion slot**. Jurafsky & Martin's *Speech and Language Processing* (free draft online) Chs. 2, 4, 6 are the gold standard.
- **Practice:** Build a sentiment classifier on movie reviews (IMDB dataset). Compare bag-of-words + logistic regression against TF-IDF + logistic regression.
- **Self-check:** You can explain why TF-IDF generally beats raw counts.

Module 19: Recommendation Systems

- **Concepts:** Content-based filtering, collaborative filtering (user-user and item-item), matrix factorization. Cold-start problem.
- **Reading:** **Companion slot** — Aggarwal's *Recommender Systems: The Textbook* if you want depth, or just an applied tutorial.
- **Practice:** Build a movie recommender on the MovieLens 100k dataset using both a content-based approach and matrix factorization (SVD via `surprise` library).
- **Self-check:** You can explain why Netflix doesn't use simple "users like you also watched" anymore.

Module 20: Ensemble Techniques

- **Concepts:** Bagging, Random Forests, boosting (AdaBoost, Gradient Boosting, XGBoost, LightGBM). Why ensembles work (variance reduction for bagging, bias reduction for boosting).
- **Reading:** PSDS Ch. 6 — bagging, random forest, and boosting sections.
- **Practice:** On the dataset from Module 17, add Random Forest and XGBoost. Tune them. They should outperform every previous model — by how much?
- **Self-check:** You can explain why XGBoost has dominated tabular Kaggle competitions for the past decade.

Module 21: Deep Neural Networks I

- **Concepts:** The perceptron, multilayer perceptrons. Activation functions (ReLU, sigmoid, softmax). Backpropagation conceptually. Training loops in PyTorch (`nn.Module`, `forward`, `loss.backward()`, `optimizer.step()`).

- **Reading: Companion slot** — primary: *Deep Learning with PyTorch* by Stevens, Antiga & Viehmann (free PDF from Manning) Chs. 1–6, or Sebastian Raschka's *Machine Learning with PyTorch and Scikit-Learn* Chs. 11–13. For theory backstop: the free online *Deep Learning* book by Goodfellow et al. Géron's *Hands-On ML* Chs. 10–11 are still useful conceptually but written in Keras — read for ideas, code in PyTorch.
- **Practice:** Train a feedforward neural network on MNIST in PyTorch. Write the training loop yourself (don't use a high-level wrapper like Lightning yet — you need to see `zero_grad()`, `backward()`, and `step()` to understand what they do). Get to >97% test accuracy.
- **Self-check:** You can name three activation functions and explain when to use each.

Module 22: Deep Neural Networks II

- **Concepts:** Convolutional networks (CNNs) for images. Recurrent networks (RNN, LSTM) for sequences at a conceptual level. Transfer learning.
 - **Reading:** *Deep Learning with PyTorch* (Stevens et al.) Chs. 7–8 for CNNs; Raschka's *ML with PyTorch* Chs. 14–16 for CNNs and RNNs. PyTorch's own tutorials at pytorch.org/tutorials are excellent and version-current — use them alongside the books.
 - **Practice:** Train a CNN on CIFAR-10 in PyTorch. Then redo the task with transfer learning from a pretrained model (`torchvision.models.resnet50` with ImageNet weights). Compare accuracy and training time.
 - **Self-check:** You can explain why convolutions are "translation-invariant" and why that's useful for images.
-

PART 4 — Generative AI and Capstone (Weeks 23–26)

Module 23: Introduction to Generative AI

- **Concepts:** Transformer architecture at a conceptual level (attention, self-attention). Pretraining vs. fine-tuning. Prompt engineering. RAG (retrieval-augmented generation). Limitations: hallucination, context window, cost, evaluation.
- **Reading: Companion slot** — Sebastian Raschka's *Build a Large Language Model (From Scratch)* is excellent if you want the mechanics; Chip Huyen's *Designing ML Systems* for the engineering side.
- **Practice:** Build a small RAG application: embed a folder of your own documents, store in a vector database (Chroma or FAISS), and use an LLM API to answer questions over them.

- **Self-check:** You can explain why "attention is all you need" was a paradigm shift, and what attention computes.

Module 24: Capstone Project (Weeks 24–26)

The capstone is the most important deliverable in the program. Treat it accordingly.

Project criteria:

1. **Real problem.** Not a tutorial dataset. Something from your work, an industry you care about, or a publicly available dataset that maps to a genuine question.
2. **End-to-end.** Data acquisition → cleaning → EDA → modeling → evaluation → communication.
3. **One non-obvious decision.** Why this model? Why this metric? Why this preprocessing? You should have a defensible answer to at least one hard question.
4. **Public artifact.** GitHub repo with a polished README, a notebook walkthrough, and ideally a 2-page PDF summary or a short demo video.

Three weeks of structure:

- Week 24 — Scope, find data, do EDA, write a one-page proposal.
 - Week 25 — Build models, iterate, document failures honestly.
 - Week 26 — Polish, write up, deploy if applicable, present to a friend out loud (this catches more issues than re-reading ever will).
-

How to integrate your other books

I built the outline assuming PSDS as your primary stats reference. The "companion slots" above mark places where PSDS is thin. Common books that fill these gaps well:

- **An Introduction to Statistical Learning (ISLR)** — covers most of Modules 6–14 with more depth than PSDS and free online. If you have it, use it as your second pass on every Part 2 module.
- **Hands-On ML with Scikit-Learn, Keras & TensorFlow (Géron)** — best applied companion for the **classical ML** modules: 8, 9, 15, 16. The Scikit-Learn half of the book (Chs. 1–9) is framework-agnostic and excellent. *Note: the deep learning chapters use Keras/TensorFlow — for those modules use a PyTorch-native book instead (below).*
- **Deep Learning with PyTorch (Stevens, Antiga & Viehmann)** — free PDF from Manning. Primary text for Modules 21 and 22.

- **Machine Learning with PyTorch and Scikit-Learn (Raschka, Liu & Mirjalili)** — strong alternative or supplement for Modules 21-22, with broader coverage of training tricks and architectures. Raschka also has free companion notebooks on GitHub.
- **PyTorch official tutorials (pytorch.org/tutorials)** — version-current, free, and indispensable. Always cross-check book code against these since the API evolves.
- **Forecasting: Principles and Practice (Hyndman)** — free online; the right book for Module 10.
- **Speech and Language Processing (Jurafsky & Martin)** — free online; right book for Module 18.
- **Deep Learning (Goodfellow, Bengio, Courville)** — free online; theory backstop for Modules 21-22.

If you tell me which books you actually have, I'll do a chapter-by-chapter mapping into this outline so each module has a primary read, a companion read, and a practice task.

Tracking your progress

Keep one master GitHub repo named something like `ml-ai-self-study` with a folder per module. In each folder: the notebook, a `README.md` with what you learned, and the dataset (or a link to it). By Module 24 you'll have 24 notebooks plus a capstone — a stronger portfolio than most bootcamp graduates produce.

Good luck.